

serve-sim + simstream vs stock serve-sim: results

2026-09-25. Fork: `expo-device-hub-simstream`, branch `simstream-video`.

Summary

We replaced serve-sim's video path with the simstream engine and kept serve-sim's UI, input and tools. Against serve-sim's own two video modes, the simstream path:

- **Sends 3.5–4.2× the pixels at the same latency as WebRTC.** It streams at full native resolution (1206×2622 on the iPhone 16 Pro, 1320×2868 on the 17 Pro Max). Stock WebRTC streams at 640×1392, and its latency is still no lower.
- **Holds 60 fps.** In the controlled benchmark it showed 59.8–59.9 distinct frames per second and skipped 0.2% of frames. Stock WebRTC showed 48–51 fps and skipped 19–26%.
- **Never froze.** There were 0 freezes over 50 ms in 160 s of measurement, against 20 for WebRTC and 32 for HTTP/AVCC.
- **Has about 7× lower latency than serve-sim's HTTP/AVCC mode** (38–42 ms against 273–283 ms).
- **Found a serve-sim bug that affects every mode proxied through Node.** The fix is included.

Where it **didn't** win: the over-the-network route recording shows simstream and WebRTC tied on touch-to-screen time and on frames shown. On latency alone, the two are at parity; the difference is what each delivers at that latency (resolution, pacing, no freezes).

1. Controlled latency benchmark (same machine)

Setup:

- **Machine:** M4 Max, iPhone 16 Pro simulator on iOS 18.6.
- **Scene:** a barcode clock page in the simulator. Headless Chrome decodes the barcode from whatever the viewer page displays and records how old the displayed frame is.
- **Runs:** 4 runs of 20 s per mode and scene. The mode order was rotated each rep to cancel drift.

Light scene	serve-sim + simstream	stock WebRTC	stock HTTP/AVCC
Frame age, mean	37.6 ms	36.0 ms	272.6 ms
Frame age, p95	43.5 ms	44.8 ms	298 ms
Distinct fps shown	59.9	50.6	54.1
Source frames skipped	0.2%	18.6%	6.0%
Freezes over 50 ms (80 s total)	0	12	16
Worst gap	34 ms	68 ms	257 ms
Resolution	1206×2622	640×1392	1206×2622

Heavy scene	serve-sim + simstream	stock WebRTC	stock HTTP/AVCC
Frame age, mean	41.6 ms	38.8 ms	283.2 ms
Frame age, p95	44.0 ms	50.2 ms	320 ms
Distinct fps shown	59.8	48.2	53.2
Source frames skipped	0.2%	26.2%	8.4%
Freezes over 50 ms (80 s total)	0	8	16
Worst gap	50 ms	68 ms	316 ms

WebRTC's mean latency is 1.6–2.8 ms lower. It gets there by encoding a quarter of the pixels, and it drops a fifth to a quarter of the frames. simstream's p95 matches or beats it.

Engine timing (simstream at full resolution): capture to encoder 1.3 ms, encode 8.4 ms, network queue about 1 ms.

2. Over the network

Setup:

- **Server:** serve-sim on the Mac Mini (M4 Pro, iPhone 17 Pro Max on iOS 26.5).
- **Viewer:** headless Chrome on an off-prem MacBook, about 13 ms away over a direct Tailscale path.
- **Route:** the same scripted 24-input route (Settings → General → About → back → home → Calendar year/month/day/multi-day). Every touch was sent from the viewer page, so video and input both crossed the network.
- **Measurement:** touch to screen is the time from the viewer dispatching a touch to the viewer's screen first changing. It includes the app's own reaction time, which is identical across modes.

	serve-sim + simstream	stock WebRTC	stock HTTP/AVCC
Touch to screen, median	100 ms	100 ms	300 ms
Touch to screen, mean	101 ms	97 ms	286 ms
Touch to screen, p90	183 ms	183 ms	417 ms
Distinct frames in the 1 s after each touch (sum of 24)	868	862	698
Resolution	1320×2868	640×1392	1320×2868
Recorder stalls	0	0	0

- **Network result:** simstream and WebRTC are tied on responsiveness and smoothness. simstream carries $4.2\times$ the pixels while doing it. HTTP/AVCC is about $3\times$ slower to respond.
- **On the same machine** (the same route on the M4), simstream showed 838 frames in the 1 s after touches, against 754 for WebRTC and 709 for AVCC.

Why: the architectural differences

- **Capture is locked to the simulator's render clock.** simstream encodes a frame when the simulator produces one (damage callbacks on the framebuffer surface). serve-sim's WebRTC path samples the framebuffer on its own timer, which drifts against the simulator's 60 Hz. Some frames are captured twice and others never, hence the 19–26% skipped.
- **Nothing queues.** simstream has one encoder per viewer, and the viewer acks every frame. Bitrate control is delay-based, from those acks, and a stall pauses encoding instead of piling frames up. serve-sim's HTTP/AVCC path has a 16-frame native queue that stays full, which is about 270 ms of standing latency.
- **Full resolution with no downscaling.** VideoToolbox low-latency H.264 encodes the full frame in about 8 ms. Upstream caps H.264 over WebRTC at 1280 on the long edge; per its own source, full resolution “stalls and does not recover”. In practice, the stream came out at 640×1392.

- **Presentation is paced per refresh.** Decoded frames are drawn immediately, at most one per display refresh. A burst after a hiccup collapses to the newest frame instead of fast-forwarding.

The serve-sim bug we found (fix included)

serve-sim's `/grid/api/memory` endpoint is polled by its UI every 5 s. It ran `ps -axo rss=,args=,sysctl` and `vm_stat` with `execSync`, blocking Node's event loop for about 75 ms each time. Every stream proxied through the process (HTTP/AVCC, MJPEG, and our socket) froze for that long every 5 s. Switching to async `execFile` (commit `6512155`) took simstream through serve-sim from 16 freezes to 0 per 80 s of measurement. That matches simstream standalone, and the fix applies upstream on its own.

Caveats

- **Machine load:** the M4 was busy during parts of the benchmark (other agents, Docker). Runs were interleaved so load affects all modes alike. Two heavy reps ran at a load around 70.
- **Stock WebRTC resolution:** the 640×1392 is upstream's default behavior at commit `ec75fe7`. The published 0.3.4 release ran WebRTC at 1206×2622 or 804×1748 in our earlier test, with similar latency and similar frame skipping (18%).
- **Network latency:** the barcode latency can't be measured across machines (the two clocks differ), so the network comparison uses touch to screen from the recordings. That's resolved to one frame (16.7 ms) and includes the app's reaction time.
- **Sample size:** the network route is one recording per mode (24 touches each). The latency benchmark is 4×20 s per mode and scene.

Videos

Tailnet only: [https://seth-webster-m4.\\$SIMSTREAM_TAILNET:8450/](https://seth-webster-m4.$SIMSTREAM_TAILNET:8450/)

- **Over the network:** Mini to off-prem MacBook, touches drawn, every input frame-aligned.
- **Same machine:** the M4, touches drawn.
- **Latency clock scene:** live frame-age captions.